UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026

Applied Parallel Programming

Lecture 12: Overview of CNN and GPT-2 Projects

Hrishi Shah

02/26/2026

Course Logistics

Project Milestone 1 is due Friday, March 6th, 2025.

PLEASE PULL THE PROJECTS AND START WORKING!!!

The 3-day late deadline applies for all project deliverables.

Do not wait until the last second and expect Delta to be 100% available for you to test code.

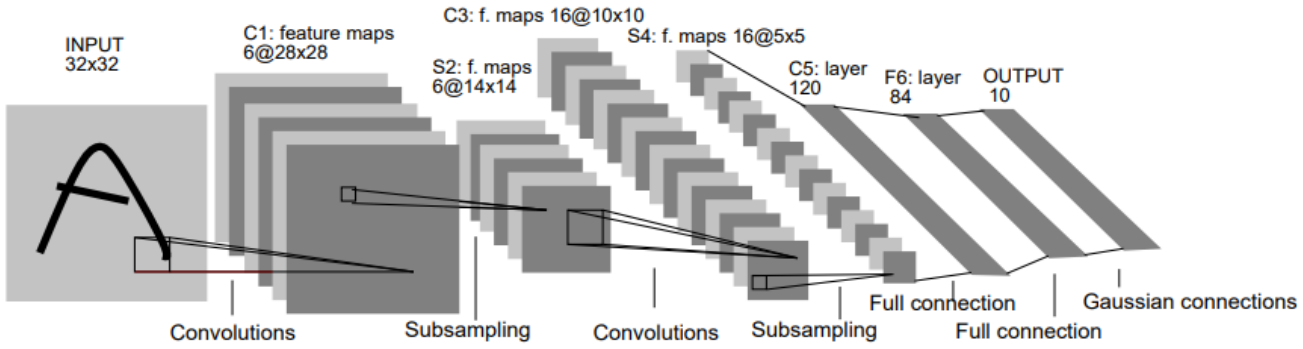
Exam next Tuesday, March 3rd, 7:00 PM, check Canvas for details.
Review session this Saturday, February 28th, 3:00 PM to 5:30 PM at ECEB 1015

Project Release

- **Project Milestone 1 is already released (only first milestone for now)**
- README.md and its supporting documents contain all the details and instructions on how to complete the project
- The Github repo will be updated to contain additional information for PM2 and PM3
- Future milestones (for both projects) may have a PrairieLearn and/or a Gradescope component

CNN Project Overview

- Optimize the forward pass of the convolutional layers in a modified LeNet-5 CNN using CUDA. (CNN implemented using Mini-DNN, a C++ framework)
- The network will be classifying Fashion MNIST dataset
- Some network parameters to be aware of
  - Input size: 86x86 for first call, 40x40 for second call
  - Batch of 100-10k images
  - Input channels: 1 for first call, 4 for second call
  - Map out features: 4 for first call, 16 for second call
  - Convolutional kernel size: 7x7



CNN Project Timeline

- **All milestones are due on Fridays at 8PM Central Time**

- Everyone must individually submit all milestones
  - **No code sharing is allowed.**

- Project Milestone 1
  - CPU and GPU implementations of conv. layer
- Project Milestone 2
  - Profiling, matrix unrolling and kernel fusion for matrix unrolling (check last lecture)
- Project Milestone 3
  - Kernel optimizations for convolution and matrix unrolling

Natural Language Processing

- Language is a collection of symbols with certain arrangements to convey information.

- **NLP with machine learning**
  - Speech recognition, machine translation, sentiment analysis, etc.
  - Breaks phrases or sentences into words:
    - Syntax: The arrangement of the words so that a sentence makes sense
    - Semantics: The interpretation of the words so that a sentence has meaning

- Inherently difficult due to the nature of human language

Reference: Prof. Kindratenko

Shortcomings of CNNs

Prompt:

"Hrishi was not able to visit Charles as his car broke down."

What problems would a CNN have in understanding the context of this sentence?

- Who is "his" referring to? *(A CNN with a small window might think Charles, but he's probably not the one driving based off this sentence)*
- CNNs cannot flexibly support distant words *(can raise convolution mask size, but even then, you can't dynamically adjust the mask from sentence to sentence)*
- Somehow need to support local context as well as distant words

Motivation for Transformers

Self-attention mechanism can be used to capture dependencies and features across an entire sequence of text at once (much larger than some convolution window, given that you have a high enough context window).

- Unlike static convolution mask, self-attention assigns different input elements (tokens) different weights
- Convolution layers require a fixed input size and use a fixed kernel size
- Led to the transformer model (Attention is All You Need) which removed all convolution aspects for solely attention mechanisms
- However, this has quadratic runtime scaling to the input length

Transformers are more modern, hence the motivation to introduce a new project that is more relevant.

Understanding the Context: Encoder Block

The Encoder’s job is to read an entire sequence of text all at once and build a deep, rich understanding of how every word relates to every other word.

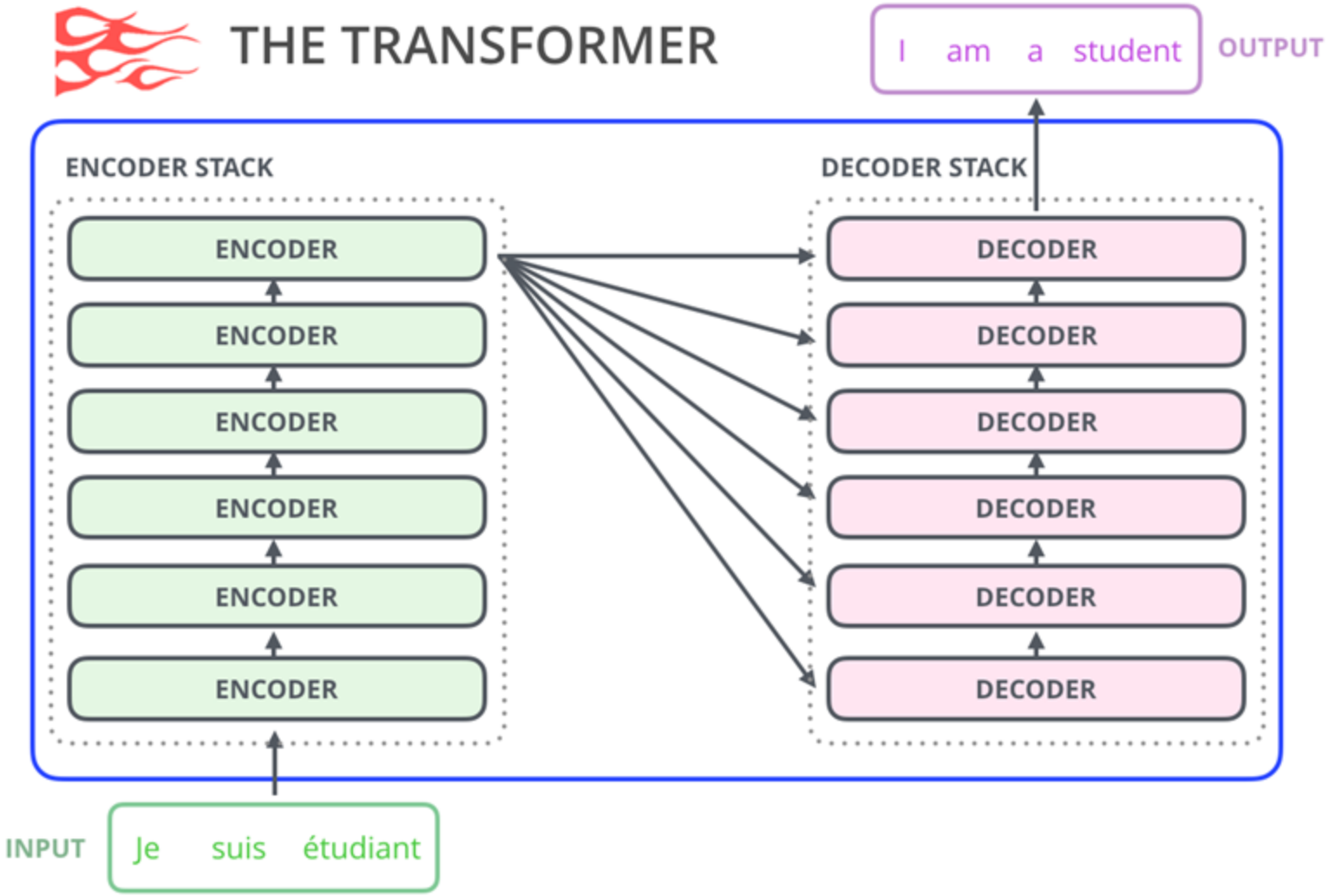
- Every token looks at every other token (past and future) simultaneously to understand context (e.g., knowing that "bank" refers to a river, not a financial institution, based on the end of the sentence).
- Residual connections and Layer Normalization to stabilize the learning process and allow gradients to flow through deep networks.
- A two-layer fully connected network (usually with a GeLU or ReLU activation) applied to each token independently to process the attention results. (FFN)

Generating the Future: Decoder Block

The Decoder’s job is to generate text autoregressively (one word at a time). It must predict the next token without "cheating" by looking ahead at the answer.

- The "Mask" hides future tokens. When predicting the 4th word, the model can only attend to words 1, 2, and 3. It cannot look right.
- In original seq2seq Transformers, the decoder has a layer to look back at the Encoder's output. (Note: GPT drops this layer!)
- Add & Norm / FFN: Similar to the encoder, these layers stabilize and transform the representations before predicting the next token.

The Original Transformer



BERT vs. GPT

BERT (Bidirectional Encoder Representations from Transformers)

- Architecture: Stacks of Encoder blocks only.
- Context: Bidirectional. It looks at the entire sentence at once (both left and right of a word).
- How it Learns: "Fill in the blank" (Masked Language Modeling). It hides a word in the middle of a sentence and tries to guess it based on surrounding context.
- Best For: Natural Language Understanding (NLU). Tasks like sentiment analysis, document classification, and reading comprehension.

BERT vs. GPT (continued)

GPT (Generative Pre-trained Transformer) — *Our Project!*

- Architecture: Stacks of Decoder blocks only.
- Context: Unidirectional (Causal). It can only look at the past words, never the future words.
- How it Learns: "Next-token prediction" (Autoregressive). It looks at a sequence of words and guesses what the very next word should be.
- Best For: Natural Language Generation (NLG). Tasks like chatbots, writing essays, summarizing text, and writing code.
- **For your project, we will give you a pretrained model (no training needed) and you solely have it generate new text**

A Transformer for Language Generation - GPT

The transformer from the aforementioned paper excels at tasks like text translation, but it is not set up for text generation.

- For language generation, we want the model to only see text before the next prediction. Should not be able to see into the future (causal)
  - Can remove the encoder stack then and replace it with a masked attention (triangular)
- Learnable position encodings, which are trained with the model rather than constant ones
- Much more layers through something called a transformer block that can be chained several times

Other minor nuances exist, yet this is at the heart of GPT.

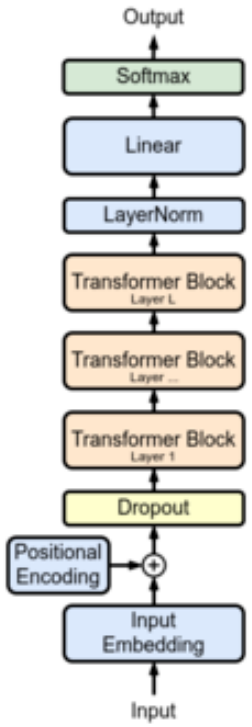
GPT Architecture Overview

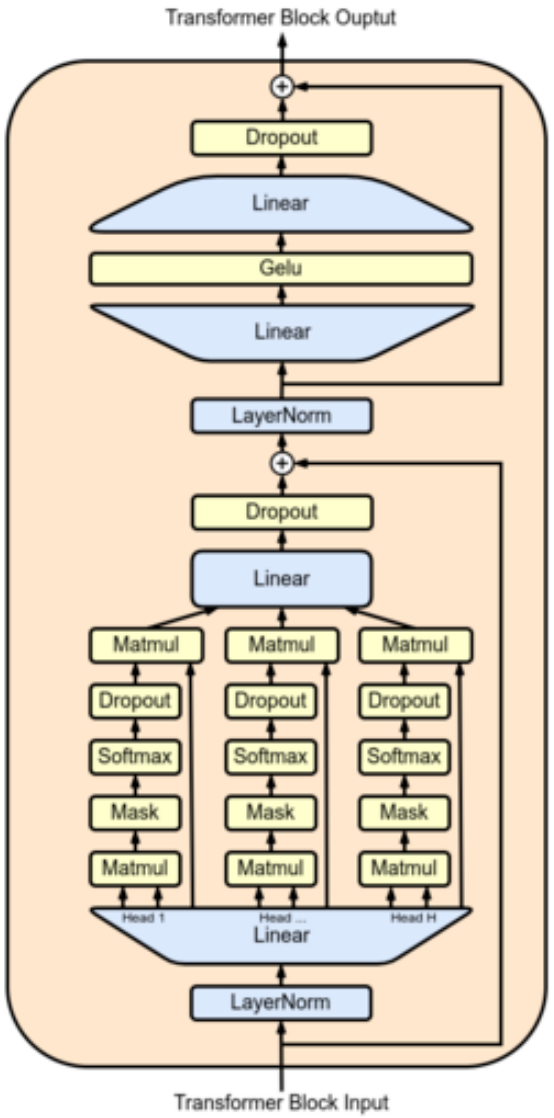
"Decoder-only" architecture

Autoregressive:
Tokens are generated once at a time and fed back into the model inputs to generate the next.

Transformer block = decoder block, can stack several in a row (how do the # of parameters scale?)

# of attention heads and the hidden size can also be scaled. (More on this later…)



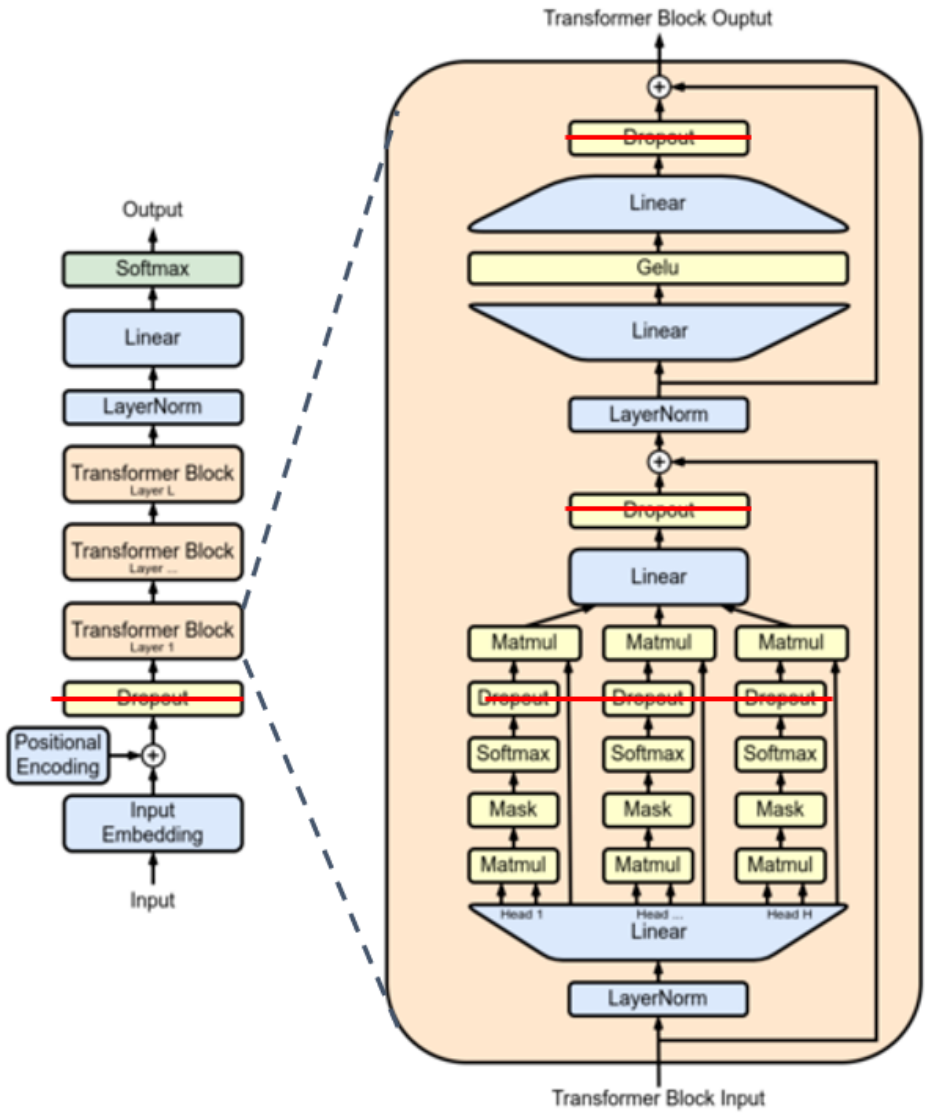


GPT Inference

Inference is the act of performing a task using a model that is already trained.

During inference, we do not care about dropout, since its only use and effects are in training.

Let us dissect GPT-2's inference process with the help of this Transformer Decoder stack diagram.



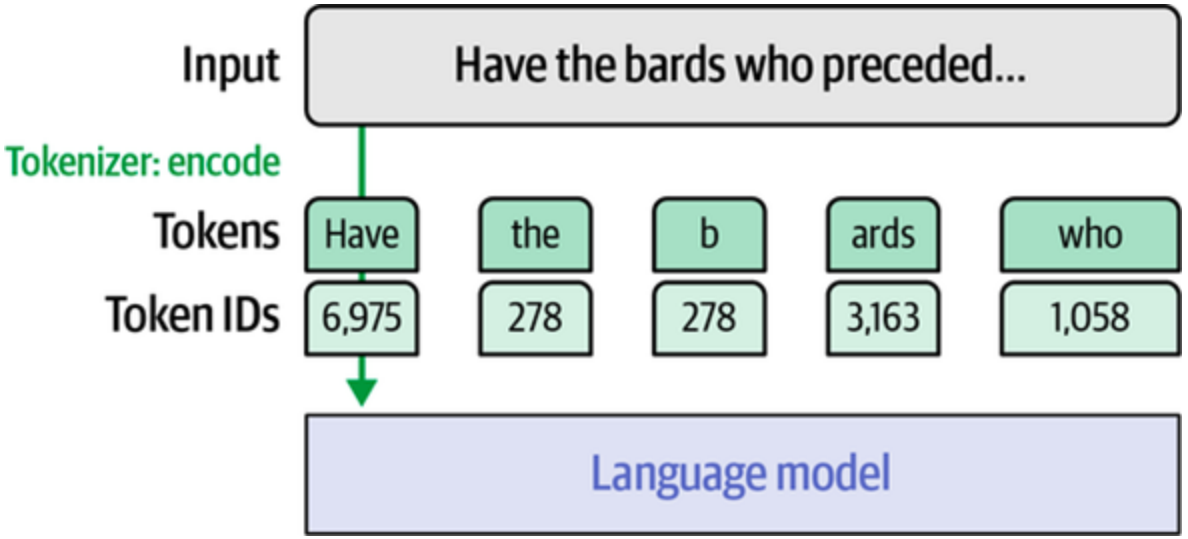
Tokenization

Operation: Split a string into smaller substrings, which each correspond to a numeric token value.

Purpose: Convert human language input into numbers for easier processing

Performance: Varies based on tokenization scheme, but usually minimal performance impact

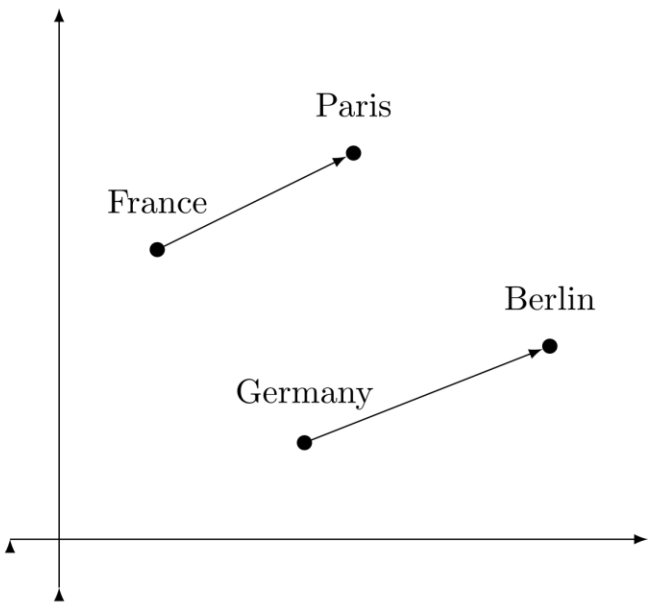
Can be quite nuanced; several complex tokenization strategies exist, and the choice can have implications on model behavior.
(i.e. GPT-2 uses byte-pair encoding, combining frequent symbols to tokens)

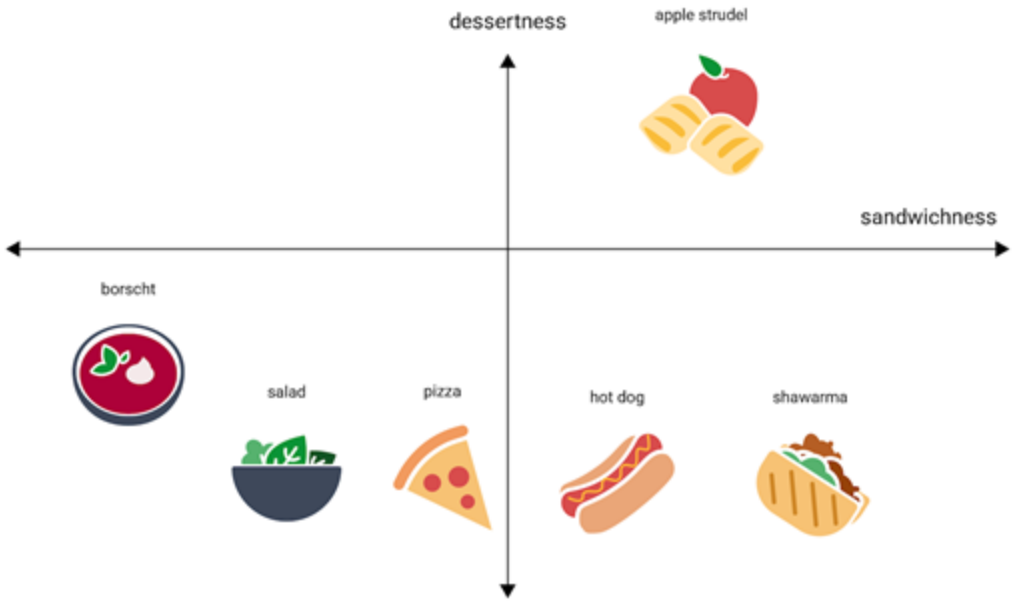


Think like LLMs: Token Embedding

In NLP, tokens and semantics are often encoded as vectors. Different directions represent different meanings, and vectors pointing in similar directions have semantic relation. This direction-meaning relationship is formed during model training.

In GPT-2 Small, tokens are embedded in a 768-dimensional space





Encoding Tokens

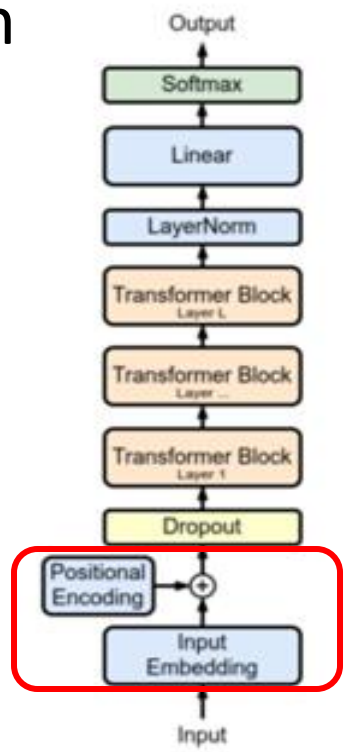
Operation: Converts a token (single value) into the token embedding space (vector) to encode its meaning, which also encodes its positional information (relative in the sentence)

Purpose: Embed the tokens into a higher dimensional space so that linear transformations and other mechanisms can work with them

(Need more information than just corresponding token IDs)

Performance: Negligible

(only performed once per forward pass)

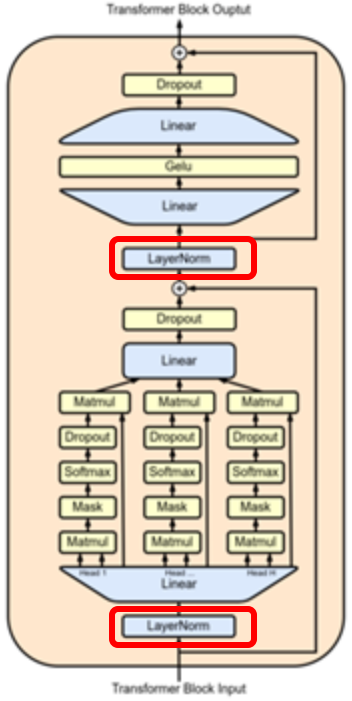


Layer Normalization

Operation: Normalizes input values using the mean and standard deviation of each row of inputs. Then scale and shift to a different mean and standard deviation.

Purpose: It prevents vanishing/exploding gradients, mitigates activation shifts, and ensures consistent input distribution across deep network layers

Performance: medium impact

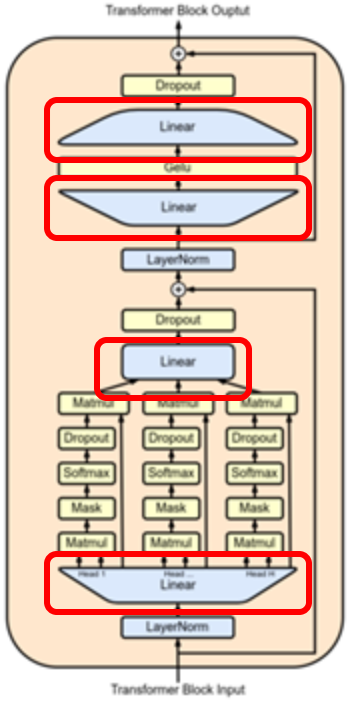


Linear Layers: Matrix Multiplication (MatMul)

Operation: Linear transformations implemented using standard matrix multiplication (with a vector addition for bias vector)

Purpose: Feature transformation, dimensionality mapping, information reinterpretation… It is used everywhere.

Performance: Significant impact

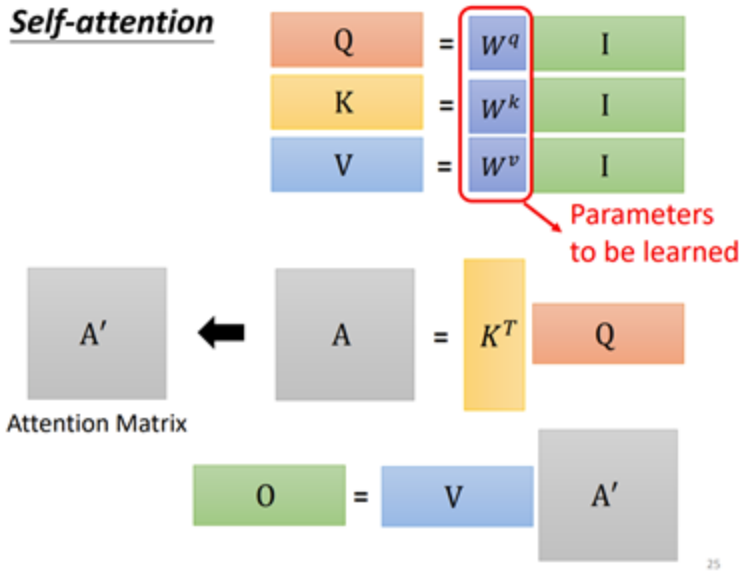


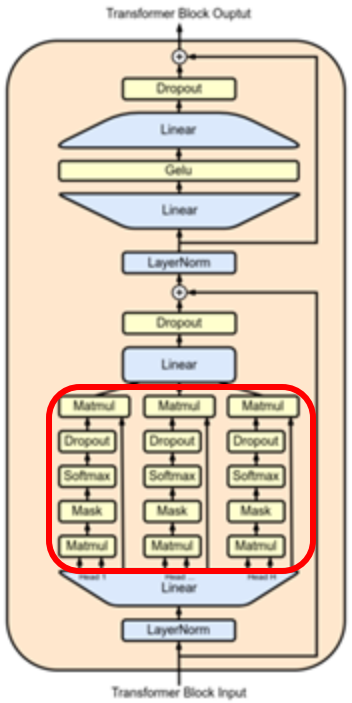
Multi-Head Self-Attention

Operation: Essentially comprises of a series of matrix multiplications. Generates key, query, and value vectors from the input, performs 2 matrix multiplications, and a softmax operation in between. (no dropout)

Purpose: Allows tokens to attend to each other, providing context across the text.

Performance: significant impact





Multi-Head Self-Attention

The query/key/value concept is analogous to retrieval systems. For example, when you search for videos on Youtube:

- **query** (text in the search bar)
- **keys** (video title, description, etc.)
- **values** (the best matched videos)

‘K’, ‘Q’ features of a token can be used as a key or a query to find the relationship with other tokens!

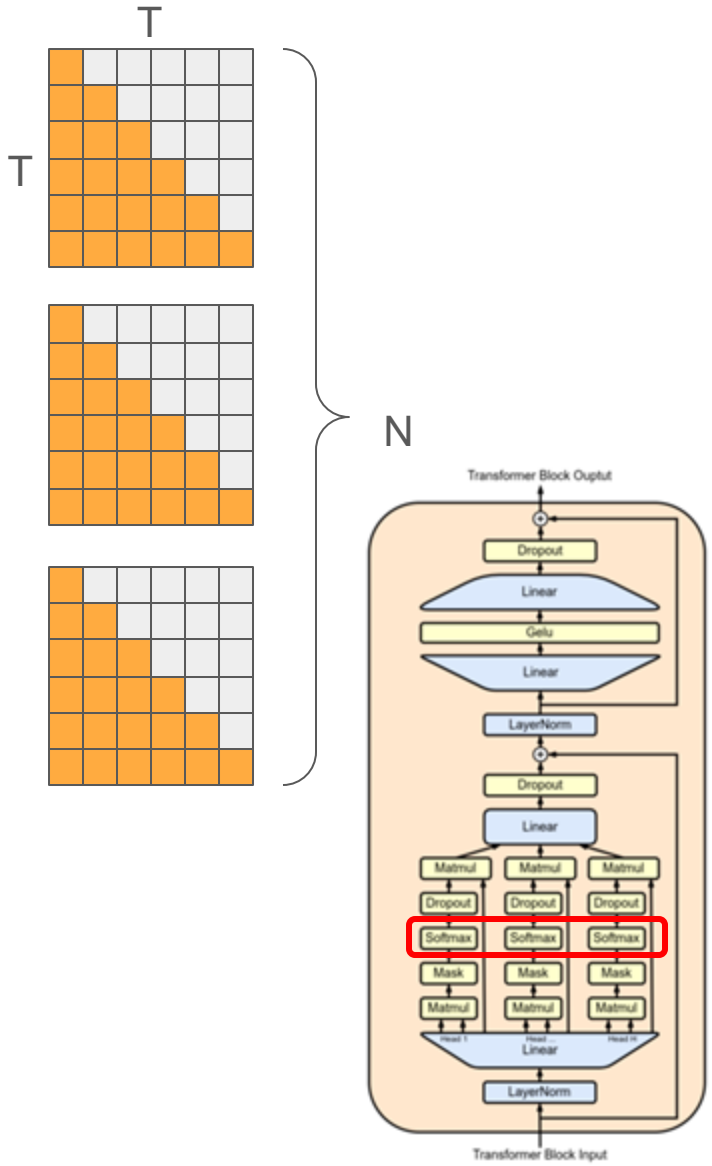
Multi-Head Self-Attention: Softmax*

Operation: Turns a list of raw numbers into probabilities.
Purpose: Used in Self-Attention. Applies causal masking to allow for autoregressive next token generation. Softmax to allow certain tokens to attend to others much more strongly.

Performance: medium impact

$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Formal Definition of Softmax, but NOT what you use in the GPT Project!

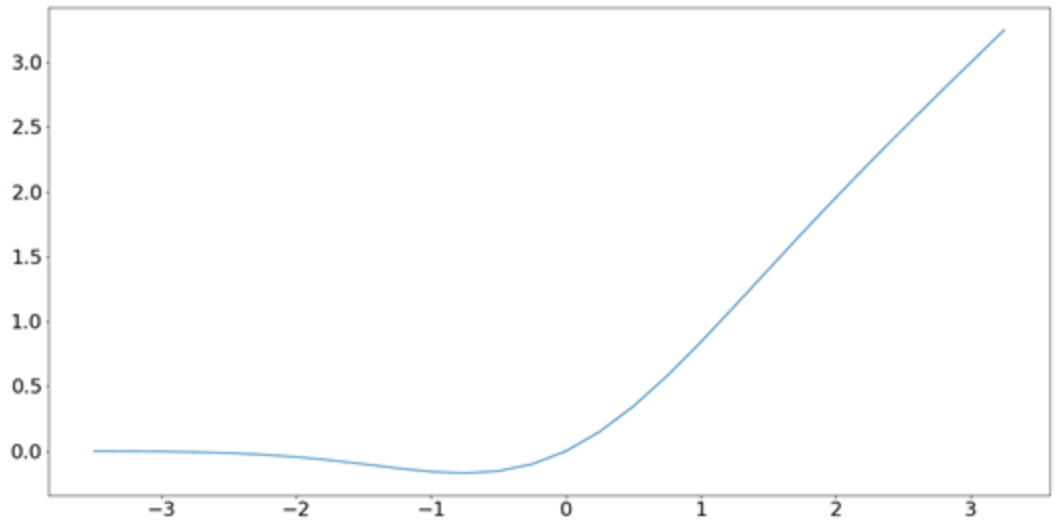


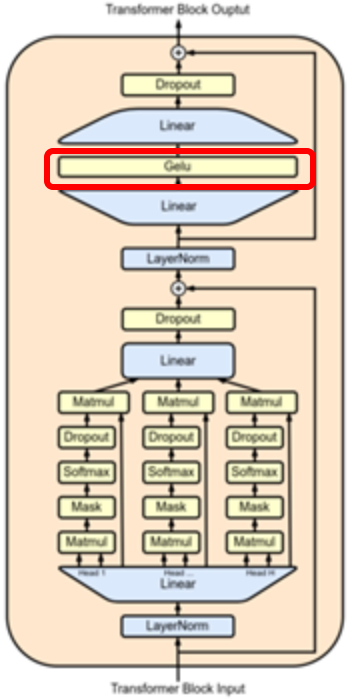
Activation Function: GeLU

Operation: Apply a Gaussian Error Linear Unit after the linear transformation to introduce non-linearity

Purpose: Used in the fully connected layers, introduces non-linearity into the model to capture more complex patterns

Performance: small impact



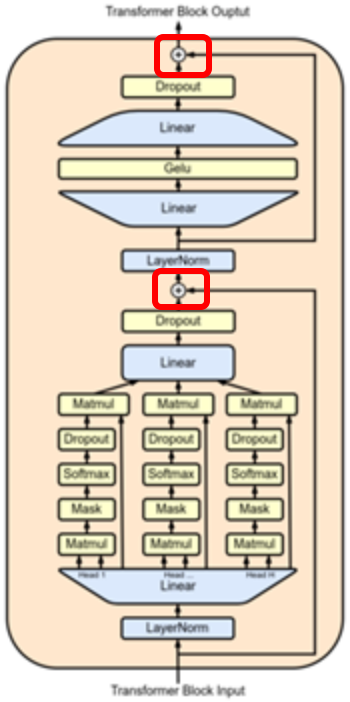


Residual Connections

Operation: Adds the Un-Attended input to the output in each layer (basic vector addition)

Purpose: Preserve previous token meaning to the next stages, good for deep networks

Performance: small impact

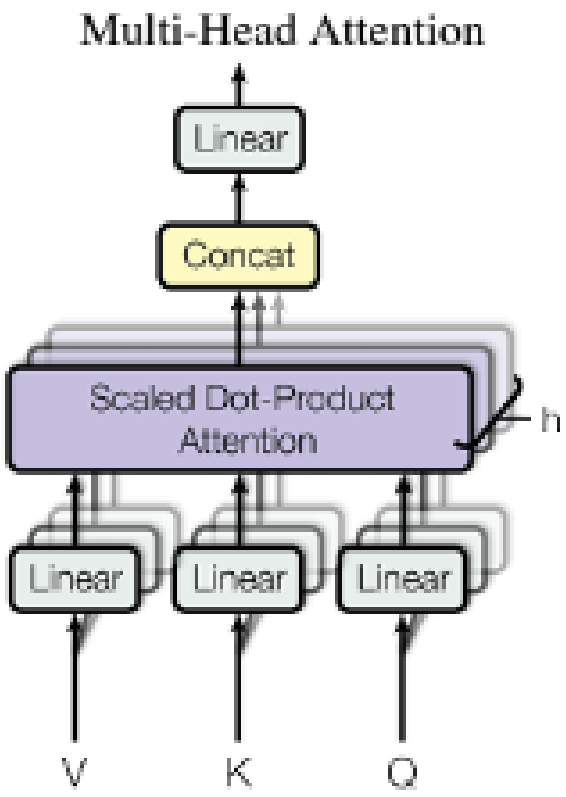


Multi-Head Attention

Diverse Representations: Multiple heads focus on various aspects of the input, creating richer context-aware representations.

Increased Expressiveness: Combining information from multiple heads captures more complex patterns and relationships.

Improved Accuracy: Multi-head attention boosts accuracy on various NLP tasks by considering multiple perspectives.



GPT Project Timeline

- All ***GPT project milestones*** are due on **Fridays at 11:59PM Central Time**

- Milestone 1
  - Implement basic GPU kernels (given CPU implementation)
  - Understand the Transformer Architecture
- Milestone 2
  - Perform both kernel and system-level profiling, propose optimizations
  - Implement several required optimizations
- Milestone 3
  - Implement more required optimizations as well as optimizations proposed in M2
  - Perform end-to-end performance analysis (final report)

- Main goal is to maximize end-to-end performance!

Questions?